CS 141 - Lab 1: Introduction to C Programming

Basic Data Types and Formatted Input/Output

Table of Contents

1. IDE Setup Instructions
2. Lab Objectives
3. Exercise 1: Hello World and Basic Output
4. Exercise 2: Working with Different Data Types
5. Exercise 3: Formatted Input and Output
6. Exercise 4: Arithmetic Operations and Formatting
7. Exercise 5: Character Input/Output
8. Challenge Exercise

IDE Setup Instructions

Option 1: Visual Studio Code (VS Code)

Step 1: Install VS Code

1. Download Visual Studio Code from https://code.visualstudio.com/
2. Install it following the installation wizard

Step 2: Install C/C++ Extension

1. Open VS Code
2. Click on the Extensions icon in the left sidebar (or press Ctrl+Shift+X / Cmd+Shift+X)
3. Search for "C/C++" by Microsoft
4. Click "Install" on the extension

Step 3: Install a C Compiler

For macOS:

- Open Terminal
- Install Xcode Command Line Tools by running: **xcode-select --install**
- This installs gcc compiler

For Windows:

- Install MinGW-w64 from https://www.mingw-w64.org/
- Or install MSYS2 which includes MinGW-w64
- Add the compiler to your system PATH

For Linux:

- Open Terminal and run: **sudo apt-get install build-essential** (Ubuntu/Debian)
- Or: **sudo yum install gcc** (CentOS/RHEL)

Step 4: Verify Installation

1. Open Terminal (macOS/Linux) or Command Prompt (Windows)
2. Type: **gcc --version**
3. You should see version information if installation was successful

Step 5: Create and Run Your First Program

1. Create a new folder for your lab work
2. Open VS Code in that folder
3. Create a file named hello.c
4. Write a simple program:

and set up run button !!
debug (gdb)

```
#include <stdio.h>

int main() {

    printf("Hello, World!\n");

    return 0;

}
```

5. Open Terminal in VS Code
6. Compile: **gcc hello.c -o hello** !!
7. Run: **./hello** (macOS/Linux) or ~~**hello.exe** (Windows)~~

~~Option 2: Code::Blocks~~

Step 1: Download and Install Code::Blocks

1. Visit http://www.codeblocks.org/
2. Download the version that includes MinGW compiler (recommended)
   - Look for "codeblocks-XX.XXmingw-setup.exe" (Windows)
   - Or download the macOS/Linux version appropriate for your system
3. Run the installer and follow the installation wizard
4. **Important:** Make sure to select the option to install MinGW compiler if prompted

Step 2: Verify Installation

1. Open Code::Blocks
2. Go to Settings > Compiler
3. Select "GNU GCC Compiler" from the dropdown
4. Click "Toolchain executables" tab
5. Verify that the compiler's installation directory is correct

Step 3: Create and Run Your First Program

1. Click File > New > Project...
2. Select "Console application" and click "Go"
3. Choose "C" (not C++) and click "Next"
4. Enter a project name (e.g., "Lab1") and choose a location
5. Click "Finish"
6. You'll see a main.c file with a template program
7. Click the green "Run" button to compile and run

Lab Objectives

By the end of this lab, you should be able to:

- Set up a C development environment using either VS Code or Code::Blocks
- Write, compile, and execute C programs
- Understand and use basic C data types (int, float, double, char)
- Use formatted input/output functions (printf, scanf) with appropriate format specifiers
- Format output using width, precision, and alignment specifiers

Exercise 1: Hello World and Basic Output

Objective: Write your first C program and understand the basic structure.

Task:

1. Create a program named exercise1.c
2. Write a program that displays the following output:

   Welcome to CS 141!

   This is my first C program.

   I'm learning about formatted output.

3. Compile and run your program
4. Make sure each line appears on a separate line

Expected Output:

```
Welcome to CS 141!

This is my first C program.

I'm learning about formatted output.
```

Hints:

- Use printf() function
- Use \n for new lines
- Don't forget #include <stdio.h> at the top

Exercise 2: Working with Different Data Types

Objective: Declare variables of different data types and display their values.

Task:

1. Create a program named exercise2.c
2. Declare variables of the following types and assign values:
   - int variable with value 42
   - float variable with value 3.14
   - double variable with value 2.71828
   - char variable with value 'A'
3. Display each variable using printf() with the appropriate format specifier:
   - %d for integers
   - %f for floats
   - %lf for doubles
   - %c for characters
4. Also display the character as an integer using %d to see its ASCII value

Expected Output Format:

```
Integer: 42

Float: 3.140000

Double: 2.718280

Character: A

Character as integer (ASCII): 65
```

Hints:

- Remember to use %d for int, %f for float, %lf for double, %c for char
- You can use %d with a char variable to see its ASCII value

Exercise 3: Formatted Input and Output

Objective: Use scanf() to read user input and display it with proper formatting.

Task:

1. Create a program named exercise3.c
2. Write a program that:
    - Prompts the user to enter their age (integer)
    - Prompts the user to enter their height in meters (float)
    - Prompts the user to enter their first initial (character)
    - Reads all three values using scanf()
    - Displays the information in a formatted way

Sample Input:

```
Enter your age: 20

Enter your height (in meters): 1.75

Enter your first initial: J
```

Expected Output:

```
You entered:

Age: 20 years old

Height: 1.75 meters

Initial: J
```

Hints:

- Use scanf("%d", &variable) for integers
- Use scanf("%f", &variable) for floats
- Use scanf(" %c", &variable) for characters (note the space before %c to consume whitespace)
- Don't forget the & (address-of operator) before variable names in scanf()

Exercise 4: Arithmetic Operations and Formatting

Objective: Perform arithmetic operations and format the output with precision.

Task:

1. Create a program named exercise4.c
2. Write a program that:
    - Reads two integers from the user
    - Performs the following operations:
        - Addition
        - Subtraction
        - Multiplication
        - Division (as float)
        - Modulus (remainder)
    - Displays all results with appropriate formatting
    - For division, display the result with 2 decimal places

Sample Input:

```
Enter first number: 15

Enter second number: 4
```

Expected Output:

```
15 + 4 = 19

15 - 4 = 11

15 * 4 = 60

15 / 4 = 3.75

15 %% 4 = 3
```

Hints:

- Use %.2f to display float with 2 decimal places
- Use %% to display a literal % character in printf
- Remember: integer division truncates, so cast to float for decimal results

Exercise 5: Character Input/Output

Objective: Work with character data type and understand ASCII values.

Task:

1. Create a program named exercise5.c
2. Write a program that:
    - Reads a single character from the user
    - Displays the character
    - Displays its ASCII value
    - Displays the next character in the alphabet
    - Displays the previous character in the alphabet
    - Displays the character in uppercase (if it's lowercase) or lowercase (if it's uppercase)

Sample Input:

```
Enter a character: m
```

Expected Output:

```
Character: m

ASCII value: 109

Next character: n

Previous character: l

Opposite case: M
```

Hints:

- Characters are stored as integers (ASCII values)
- You can perform arithmetic on characters: `char + 1` gives the next character
- To convert case, add or subtract 32 (difference between 'a' and 'A')
- Check if character is uppercase: `if (ch >= 'A' && ch <= 'Z')`
- Check if character is lowercase: `if (ch >= 'a' && ch <= 'z')`

Challenge Exercise

Objective: Combine all concepts learned to create a simple calculator program.

Task:

1. Create a program named challenge.c
2. Write a program that:
   - Displays a welcome message
   - Prompts the user to enter two numbers (floats)
   - Prompts the user to enter an operation character: '+', '-', '*', or '/'
   - Performs the requested operation
   - Displays the result in a nicely formatted way
   - Handles division by zero with an appropriate error message

Sample Input 1:

```
=== Simple Calculator ===

Enter first number: 10.5

Enter second number: 2.5

Enter operation (+, -, *, /): +
```

Expected Output 1:

```
10.50 + 2.50 = 13.00
```

Sample Input 2:

```
=== Simple Calculator ===

Enter first number: 10

Enter second number: 0

Enter operation (+, -, *, /): /
```

Expected Output 2:

```
Error: Division by zero is not allowed!
```

Hints:

- Use a switch statement or if-else to handle different operations
- Check if the second number is zero before performing division
- Format all numbers to 2 decimal places for consistency

Submission Guidelines

1. Save all your programs in the week1 folder
2. Name your files exactly as specified: exercise1.c, exercise2.c, exercise3.c, exercise4.c, exercise5.c, and challenge.c
3. Make sure all programs compile without errors
4. Test each program with the provided sample inputs
5. Include comments in your code explaining what each section does
6. Submit all files on moodle (in-class assignment).

Additional Resources

- C Format Specifiers Reference:

    - %d - integer (decimal)
    - %f - float
    - %lf - double
    - %c - character
    - %s - string
    - %.2f - float with 2 decimal places
    - %5d - integer with minimum width of 5
    - %-10s - string left-aligned in 10-character field

- Common Escape Sequences:

    - \n - newline
    - \t - tab
    - \\ - backslash
    - \" - double quote
    - \' - single quote

Good luck with your lab! If you have any questions, don't hesitate to ask.